



Université Abdelmalek Essaadi
Ecole Nationale des Sciences
Appliquées Al Hoceima



Compte rendu – TP6 JEE :

Projet Spring MVC, IOC, JSP & JSTL



Réalisé par : Hajar Elyazri

Filière : TDIA2

Encadré par : Pr. Mohammed cherradi

Année universitaire : 2025/2026

1. Introduction

1.1 Objectif du projet

Ce projet a pour objectif de développer une application web dynamique de gestion des produits en utilisant le framework Spring **MVC**.

L'application permet de réaliser les opérations **CRUD** fondamentales sur des produits :

- Afficher la liste des produits
- Ajouter un produit
- Modifier un produit
- Supprimer un produit
- Rechercher un produit par ID

1.2 Technologies utilisees

- Spring MVC : framework de presentation web
- Spring IOC : conteneur d'injection de dependances
- JPA / Hibernate : couche de persistance des donnees
- JSP (Java Server Pages) : vues de l'interface utilisateur
- JSTL : balises JSP standard (c:forEach, c:if...)
- MySQL : base de donnees relationnelle
- Apache Tomcat : serveur d'application
- Maven : outil de gestion des dependances

1.3 Donnees manipulees

Chaque produit contient les informations suivantes :

- id : identifiant unique (genere automatiquement par la BDD)
- nom : nom du produit
- description : description du produit
- prix : prix du produit

1.4 Comparaison : TP precedent (Servlets) vs Spring MVC

Ce projet remplace l'architecture multi-Servlets par une architecture Spring MVC centralisee.

Critere	TP precedent (Servlets)	Ce projet (Spring MVC)
Point d'entree	N Servlets (une par action)	1 DispatcherServlet (Front Controller)
Mapping URL	web.xml : un <servlet-mapping> par action	@RequestMapping sur chaque methode
Logique controller	Eparpillee dans chaque Servlet	Centralisee dans ProduitController
Injection dependances	Singleton manuel getInstance()	Spring IOC (@Autowired ou <property>)
Vues JSP	Accessibles par URL directe	Resolues par InternalResourceViewResolver
Configuration	Tout dans web.xml	spring-beans.xml + application-servlet-config.xml
Data binding	req.getParameter() par champ	Automatique : Produit p en parametre

2. Architecture du projet

2.1 Architecture 3-tiers et Spring MVC

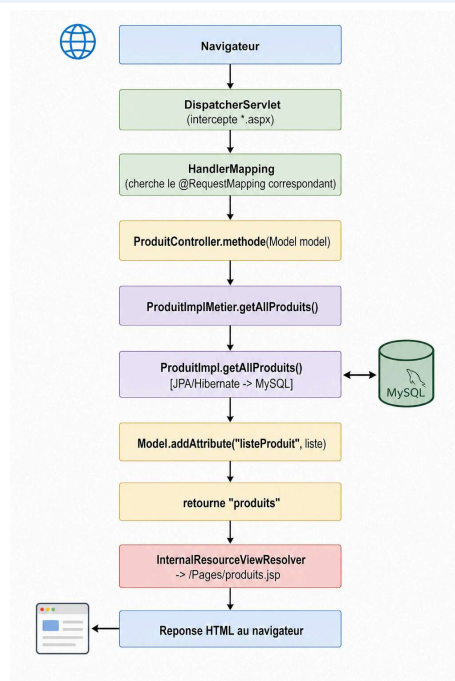
Le projet suit l'architecture 3-tiers couplée au patron MVC :

Model (M): Les classes de la couche DAO (Produit, ProduitDAO, ProduitImpl). Representent les donnees et la logique d'accès à la BDD.

View (V): Les fichiers JSP dans /Pages/. Affichent les donnees sans aucune logique métier.

Controller (C): ProduitController.java. Reçoit les requêtes HTTP, appelle le Service, prepare le Model, retourne le nom de la vue.

Flux d'une requête Spring MVC :



2.2 Structure des packages

Couche	Package / Fichier	Role
Model	dao.Produit	Entite JPA mappee sur la table produit
DAO Interface	dao.ProduitDAO	Contrat CRUD (programmer vers une interface)
DAO Impl	dao.ProduitImpl	Implementation JPA/Hibernate des operations
Util	dao.JpaUtil	Singleton EntityManagerFactory
Startup	dao.AppStartupListener	Initialise les donnees au demarrage
Service	services.ProduitMetier	Interface de la couche metier
Service Impl	services.ProduitImplMetier	Logique metier, delegation au DAO
Controller	controller.ProduitController	Gestion requetes HTTP, appelle le Service
Vue	Pages/produits.jsp	Interface utilisateur avec JSTL

2.3 Séparation des contextes Spring

Spring MVC utilise deux fichiers de configuration séparés :

Fichier	Charge par	Contient
spring-beans.xml	ContextLoaderListener	Bean DAO (ProduitImpl) + Bean Service (ProduitImplMetier) - IOC par XML
application-servlet-config.xml	DispatcherServlet	Scan @Controller + InternalResourceViewResolver - IOC par annotations
web.xml	Serveur Tomcat	Configuration globale : DispatcherServlet + ContextLoaderListener
persistence.xml	JpaUtil (Hibernate)	Connexion MySQL, dialecte, hbm2ddl.auto

3. Principe de l'Inversion de Contrôle (IOC)

3.1 Problème : couplage fort sans IOC

Sans IOC, le Controller cree lui-meme ses dépendances (couplage fort) :

```
// AVANT (Servlets) : Singleton manuel -> couplage fort
public class AddProduitServlet extends HttpServlet {
    // Le Controller depend directement de l'implementation concrete
    private static final ProduitMetier metier = ProduitMetierImpl.getInstance();
}
```

3.2 Solution : couplage faible avec Spring IOC

Avec Spring IOC, le conteneur injecte les dependances. Le Controller depend d'une interface, pas d'une implementation.

```
// APRES (Spring MVC) : injection par Spring -> couplage faible
@Controller
public class ProduitController {
    @Autowired // Spring injecte automatiquement le Service
    ProduitMetier services; // Interface, pas l'implementation !
}
```

3.3 Configuration IOC : deux approches combinees

Spring permet de combiner les deux types de configuration dans le meme projet :

Approche	Fichier	Utilise pour	Avantage
XML	spring-beans.xml	Bean DAO + Bean Service	Explicite, facile a comprendre pour debuter
Annotations	application-servlet-config.xml	Bean Controller	Moins de code, moderne
@Autowired	ProduitController.java	Injection du Service dans le Controller	Automatique, pas de setter

3.4 Configuration XML : spring-beans.xml

Chaque Bean est declare manuellement. L'injection se fait via <property> :

```
<bean id="objDAO" class="dao.ProduitImpl"/>

<!--
    Bean Service : couche metier.
    Injection du DAO via <property> :
        Spring appelle setDao(objDAO) sur l'instance de ProduitImplMetier.
        -> Le service connait le DAO sans jamais faire "new ProduitImpl()"
-->
<bean id="objServices" class="services.ProduitImplMetier">
    <property name="dao" ref="objDAO"/>
</bean>

</beans>
```

Move notification to No

Le setter correspondant dans ProduitImplMetier :

```
public class ProduitImplMetier implements ProduitMetier {
    private ProduitDAO dao;

    // Spring appelle ce setter et injecte l'objet ProduitImpl
    public void setDao(ProduitDAO dao) {
        this.dao = dao;
    }
}
```

3.5 Configuration par annotations :

application-servlet-config.xml

```
<context:component-scan base-package="controller"/>

<bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
    <property name="prefix" value="/Pages/" />
    <property name="suffix" value=".jsp" />
</bean>

</beans>
```

4. Implementation couche par couche

4.1 Couche Model : dao.Produit

L'entité Produit est la même que le TP précédent. Les annotations JPA remplacent le fichier de mapping XML (hbm.xml) de Hibernate classique.

```
@Entity
@Table(name = "produit")
public class Produit {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id_produit")
    private Long idProduit;

    @Column(name = "nom", nullable = false, length = 100)
    private String nom;

    @Column(name = "description", length = 255)
    private String description;

    @Column(name = "prix", nullable = false)
    private Double prix;

    // Constructeur vide requis par JPA et Spring MVC (data binding)
    public Produit() {}

    // Constructeur pratique pour créer un produit sans ID (avant persistance)
    public Produit(String nom, String description, Double prix) {}
}
```

4.2 Couche DAO : interface + implementation JPA

L'interface `ProduitDAO` definit le contrat CRUD. `ProduitImpl` l'implemente avec JPA/Hibernate (remplace l'ancienne liste en memoire).

```
// Exemple : methode getAllProduits() avec JPA
public List<Produit> getAllProduits() {
    EntityManager em = JpaUtil.getEntityManager();
    try {
        // JPQL : langage de requete oriente objet
        TypedQuery<Produit> query = em.createQuery(
            "SELECT p FROM Produit p", Produit.class);
        return query.getResultList();
    } finally {
        em.close(); // Toujours fermer l'EntityManager
    }
}
```

Gestion des transactions pour les operations d'ecriture (add, update, delete) :

```
public void addProduit(Produit p) {
    EntityManager em = JpaUtil.getEntityManager();
    try {
        em.getTransaction().begin();
        em.persist(p); // INSERT INTO produit ...
        em.getTransaction().commit();
    } catch (Exception e) {
        em.getTransaction().rollback(); // Annuler si erreur
        throw new RuntimeException("Erreur ajout", e);
    } finally {
        em.close();
    }
}
```

4.3 Couche Service : `ProduitImplMetier`

La couche service sert d'intermediaire entre le Controller et le DAO. Elle peut contenir des regles metier (validation, calculs...).

Difference cle avec le TP precedent : plus de Singleton manuel. Spring gere le cycle de vie du Bean.

```
// AVANT (TP precedent) : Singleton fait a la main
private static ProduitMetierImpl instance;
public static synchronized ProduitMetierImpl getInstance() {
    if (instance == null) instance = new ProduitMetierImpl();
    return instance;
}

// MAINTENANT : Spring gere le Singleton automatiquement
// (un Bean Spring est Singleton par default)
public class ProduitImplMetier implements ProduitMetier {
    private ProduitDAO dao;
    public void setDao(ProduitDAO dao) { this.dao = dao; }
    // ...delegue au DAO
}
```

4.4 Couche Controller : `ProduitController`

Un seul Controller regroupe toutes les actions. Chaque methode est mappee sur une URL via `@RequestMapping`.

Methode	URL (.aspx)	HTTP	Role
pageIndex()	/index	GET	Afficher la liste de tous les produits
searchProduct()	/searchProduct	POST	Rechercher un produit par ID
addProduct()	/addProduct	POST	Ajouter un nouveau produit
editProduit()	/editProduit?id=X	GET	Charger un produit dans le formulaire
updateProduitPost()	/updateProduit	POST	Enregistrer la modification
supprimerProduit()	/deleteProduit?id=X	GET	Supprimer un produit

Exemple : methode editProduit() avec annotations Spring MVC :

```
@RequestMapping(value = "/editProduit", method = RequestMethod.GET)
public String editProduit(Model model, @RequestParam Long id) {
    // Recuperer le produit a modifier
    Produit p = services.getProduitById(id);

    // Placer dans le Model : sera accessible dans la JSP via ${produitEdit}
    model.addAttribute("produitEdit", p);
    model.addAttribute("listeProduit", services.getAllProduits());

    // Retourner le nom de la vue (sans chemin ni extension)
    return "produits"; // -> /Pages/produits.jsp
}
```

Data binding automatique avec Spring MVC (ajout d'un produit) :

```
// Spring MVC remplit automatiquement l'objet Produit
// avec les parametres du formulaire (nom, description, prix)
// -> Plus besoin de req.getParameter("nom") !
@RequestMapping(value = "/addProduct")
public String addProduct(Model model, Produit p) {
    services.addProduit(p);
    model.addAttribute("listeProduit", services.getAllProduits());
    return "produits";
}
```

5. Fichiers de configuration

5.1 web.xml : descripteur de deploiement

web.xml configure les deux composants principaux de Spring MVC :


```

<context-param>
  <param-name>contextConfigLocation</param-name>
  <param-value>/WEB-INF/spring-beans.xml</param-value>
</context-param>

<listener>
  <listener-class>
    org.springframework.web.context.ContextLoaderListener
  </listener-class>
</listener>

```

```

<servlet>
  <servlet-name>action</servlet-name>
  <servlet-class>
    org.springframework.web.servlet.DispatcherServlet
  </servlet-class>
  <init-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>/WEB-INF/application-servlet-config.xml</param-value>
  </init-param>
  <load-on-startup>1</load-on-startup>
</servlet>

```

```

<servlet-mapping>
  <servlet-name>action</servlet-name>
  <url-pattern>*.asp</url-pattern>
</servlet-mapping>

<welcome-file-list>
  <welcome-file>default.aspx</welcome-file>
</welcome-file-list>

</web-app>

```

5.2 persistence.xml : configuration JPA/Hibernate

Identique au TP précédent. Hibernate crée automatiquement la table produit au premier démarrage grâce à `hbm2ddl.auto=update`.

```

<persistence-unit name="produitPU" transaction-type="RESOURCE_LOCAL">
  <class>dao.Produit</class>
  <properties>
    <property name="hibernate.connection.driver_class"
      value="com.mysql.cj.jdbc.Driver"/>
    <property name="hibernate.connection.url"
      value="jdbc:mysql://localhost:3306/gestion_produits"/>
    <property name="hibernate.connection.username" value="root"/>
    <property name="hibernate.connection.password" value=""/>
    <property name="hibernate.dialect"
      value="org.hibernate.dialect.MySQL5InnoDBDialect"/>
    <property name="hibernate.hbm2ddl.auto" value="update"/>
    <property name="hibernate.show_sql" value="true"/>
  </properties>
</persistence-unit>

```

5.3 pom.xml : dependances Maven

```
<dependencies>
  <!-- Spring MVC (inclut spring-core, spring-beans, spring-context) -->
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-webmvc</artifactId>
    <version>5.3.30</version>
  </dependency>

  <!-- Hibernate (implementation JPA) -->
  <dependency>
    <groupId>org.hibernate</groupId>
    <artifactId>hibernate-core</artifactId>
    <version>5.6.15.Final</version>
  </dependency>

  <!-- Driver MySQL -->
  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>8.0.33</version>
  </dependency>

  <!-- JSTL -->
  <dependency>
    <groupId>jstl</groupId>
    <artifactId>jstl</artifactId>
    <version>1.2</version>
  </dependency>

  <!-- Servlet API (fournie par Tomcat) -->
  <dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>javax.servlet-api</artifactId>
    <version>4.0.1</version>
    <scope>provided</scope>
  </dependency>
</dependencies>
```

6. Couche Vue : produits.jsp

6.1 Principe du ViewResolver

Spring MVC ne retourne jamais directement le contenu HTML. Le Contrôleur retourne un nom de vue (String), que le InternalResourceViewResolver traduit en chemin physique JSP.

Exemple: Le Contrôleur retourne "produits" -> Spring cherche /Pages/produits.jsp (prefix + nom + suffix)

6.2 Formulaire dynamique : Ajouter et Modifier

Un seul formulaire JSP gere les deux modes (ajout et modification) grace a l'expression EL `${produitEdit}` :

```
<%-- Si produitEdit existe -> mode modification, sinon -> mode ajout --%>
<form action="${produitEdit != null ? 'updateProduit' : 'addProduct'}.aspx"
      method="post">

    <%-- Champ cache : ID du produit (utilise uniquement en modification) --%>
    <input type="hidden" name="idProduit"
          value="${produitEdit != null ? produitEdit.idProduit : ''}" />

    <input type="text" name="nom"
          value="${produitEdit != null ? produitEdit.nom : ''}" />

    <input type="text" name="description"
          value="${produitEdit != null ? produitEdit.description : ''}" />

    <input type="text" name="prix"
          value="${produitEdit != null ? produitEdit.prix : ''}" />

    <%-- Bouton dynamique --%>
    <input type="submit"
          value="${produitEdit != null ? 'Mettre a jour' : 'Ajouter'}" />

</form>
```

6.3 Affichage de la liste avec JSTL

La balise JSTL `<c:forEach>` remplace les scriptlets Java (`<%...%>`) pour parcourir la liste envoyee par le Controller :

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>

<%-- ${listeProduit} est l'attribut place dans le Model par le Controller --%>
<table border="1" width="80%">
    <thead>

<tr><th>ID</th><th>NOM</th><th>DESCRIPTION</th><th>PRIX</th><th>Option</th></tr>
    </thead>
    <tbody>
    <c:forEach items="${listeProduit}" var="o">
    <tr>
        <td>${o.idProduit}</td>
        <td>${o.nom}</td>
        <td>${o.description}</td>
        <td>${o.prix}</td>
        <td><a href="deleteProduit.aspx?id=${o.idProduit}">supprimer</a></td>
        <td><a href="editProduit.aspx?id=${o.idProduit}">Modifier</a></td>
    </tr>
    </c:forEach>
    </tbody>
</table>
```

7. Guide de deployment

7.1 Prerequis

- JDK 8 ou superieur
- Apache Maven 3.x
- Apache Tomcat 9
- MySQL Server 8.x
- IntelliJ

Commande SQL pour creer la base de donnees :

```
CREATE DATABASE gestion_produits;
-- Hibernate cree automatiquement la table produit au demarrage
-- grace a : <property name="hibernate.hbm2ddl.auto" value="update"/>
```

7.3 Structure finale du projet

```
GestionDesProduits-SpringMVC/
├── pom.xml
├── src/main/
│   ├── java/
│   │   ├── controller/
│   │   │   └── ProduitController.java <- @Controller, toutes les actions
│   │   ├── dao/
│   │   │   ├── Produit.java <- @Entity JPA
│   │   │   ├── ProduitDAO.java <- Interface CRUD
│   │   │   ├── ProduitImpl.java <- Implementation JPA/Hibernate
│   │   │   ├── JpaUtil.java <- EntityManagerFactory Singleton
│   │   │   └── AppStartupListener.java <- @WebListener initialisation
│   │   └── services/
│   │       ├── ProduitMetier.java <- Interface service
│   │       └── ProduitImplMetier.java <- Implementation (injection par Spring)
│   ├── resources/
│   │   └── META-INF/
│   │       └── persistence.xml <- Config JPA/Hibernate/MySQL
│   ├── webapp/
│   │   ├── default.aspx <- Redirection vers /index.aspx
│   │   └── Pages/
│   │       └── produits.jsp <- Vue principale (JSTL)
│   └── WEB-INF/
│       ├── web.xml <- DispatcherServlet + ContextLoader
│       ├── spring-beans.xml <- Beans DAO + Service (XML IOC)
│       └── application-servlet-config.xml <- @Controller scan + ViewResolver
```

8. Conclusion

Ce projet a permis de mettre en pratique les concepts fondamentaux du developpement web Java EE avec Spring MVC.

Apports principaux

- Spring MVC : un seul DispatcherServlet remplace toutes les Servlets du TP precedent.
- IOC Spring : suppression des Singletons manuels. Spring gere le cycle de vie des Beans.
- Configuration hybride : IOC par XML pour DAO/Service, par annotations pour le Controller.
- Data binding automatique : Spring remplit les objets Java depuis les formulaires HTML.
- ViewResolver : separation claire entre la logique Controller et les vues JSP.
- JPA/Hibernate : persistance reelle en base MySQL (meme qu'au TP precedent).

Points clés à retenir

DispatcherServlet : Front Controller de Spring MVC. Intercepte toutes les requêtes et les dispatch vers le bon `@RequestMapping`.

@Controller + @Autowired : Spring détecte et injecte automatiquement les dépendances. Plus de `new()`, plus de `getInstance()`.

Model : Conteneur de données entre Controller et JSP. `model.addAttribute()` -> `${attribut}` dans la JSP.

InternalResourceViewResolver : Traduit le nom de vue retourné par le Controller en chemin JSP (prefix + nom + suffix).

spring-beans.xml : Contexte racine Spring (DAO + Service). Partagé entre toute l'application.

application-servlet-config.xml : Contexte du DispatcherServlet (Controller + ViewResolver). Chargé par le DispatcherServlet uniquement.